GeeksforGeeks

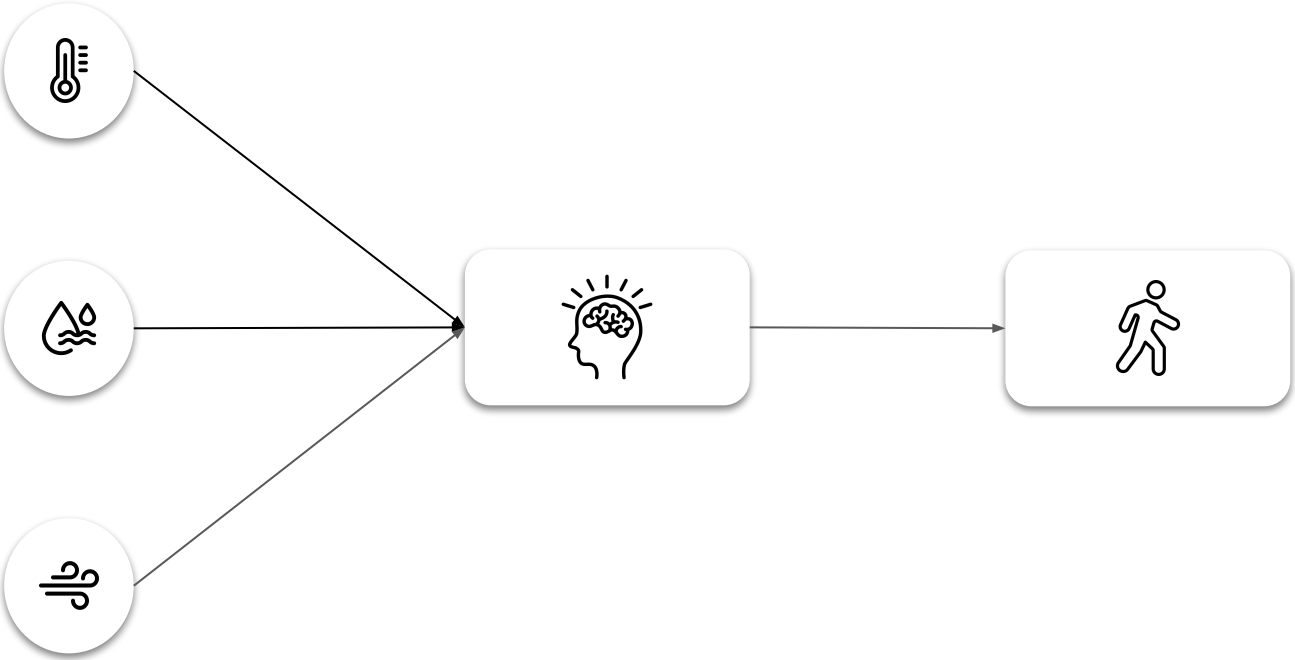
The Perceptron

The Maths of Artificial Neural Network

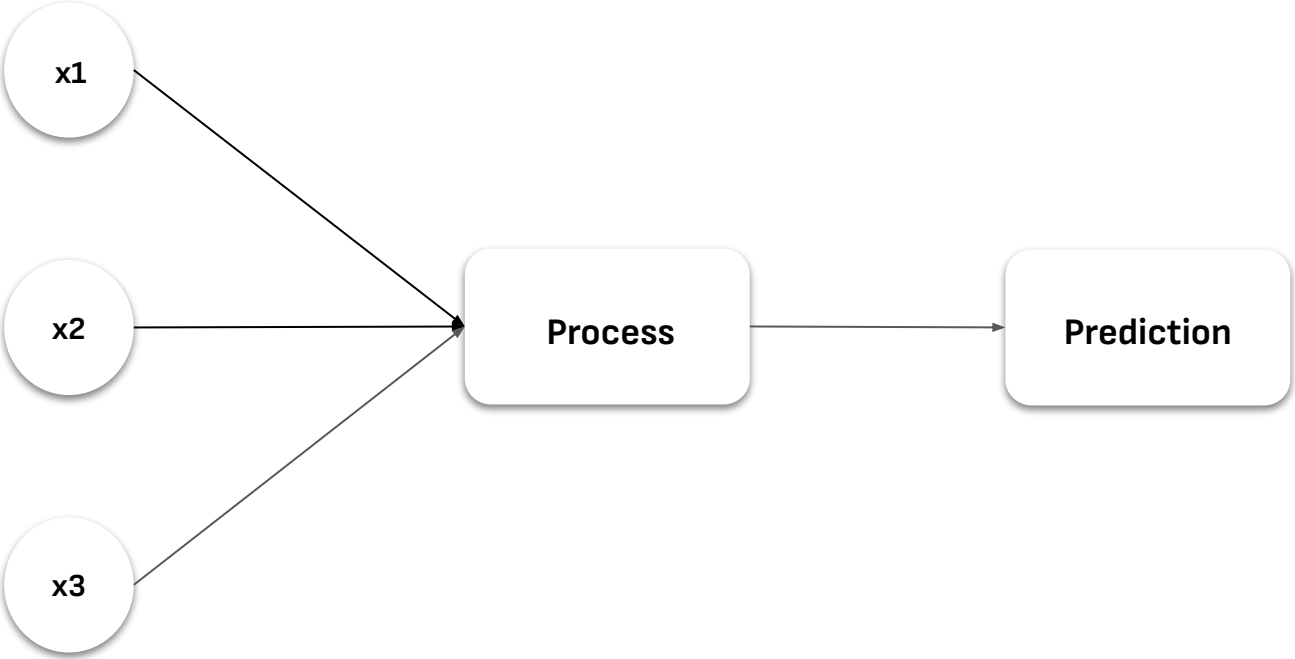
The Perceptron is a tiny decision making unit

Inspired by the brain's biological unit - the neuron

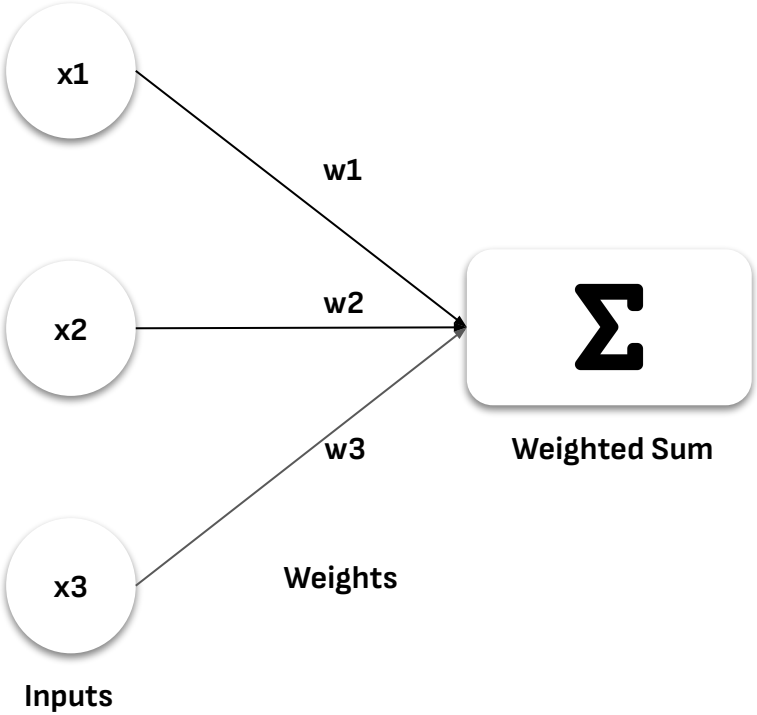
Imagine you are deciding whether to go outside. Your brain takes inputs like temperature, rain, wind speed, etc. Processes them, and makes a decision.



The Perceptron behaves the same way

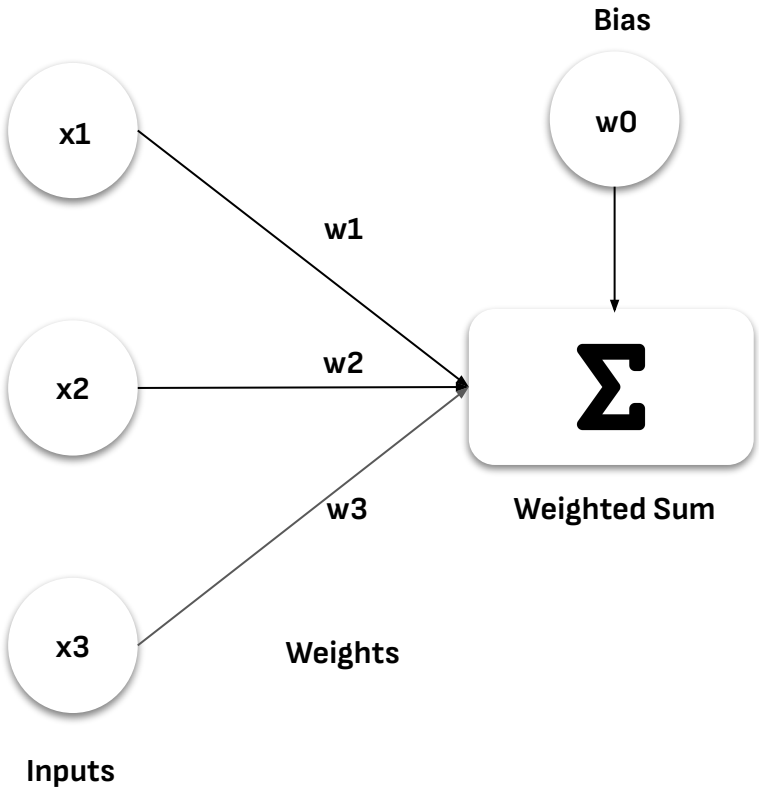


The impact of each input on the final prediction is represented by weights



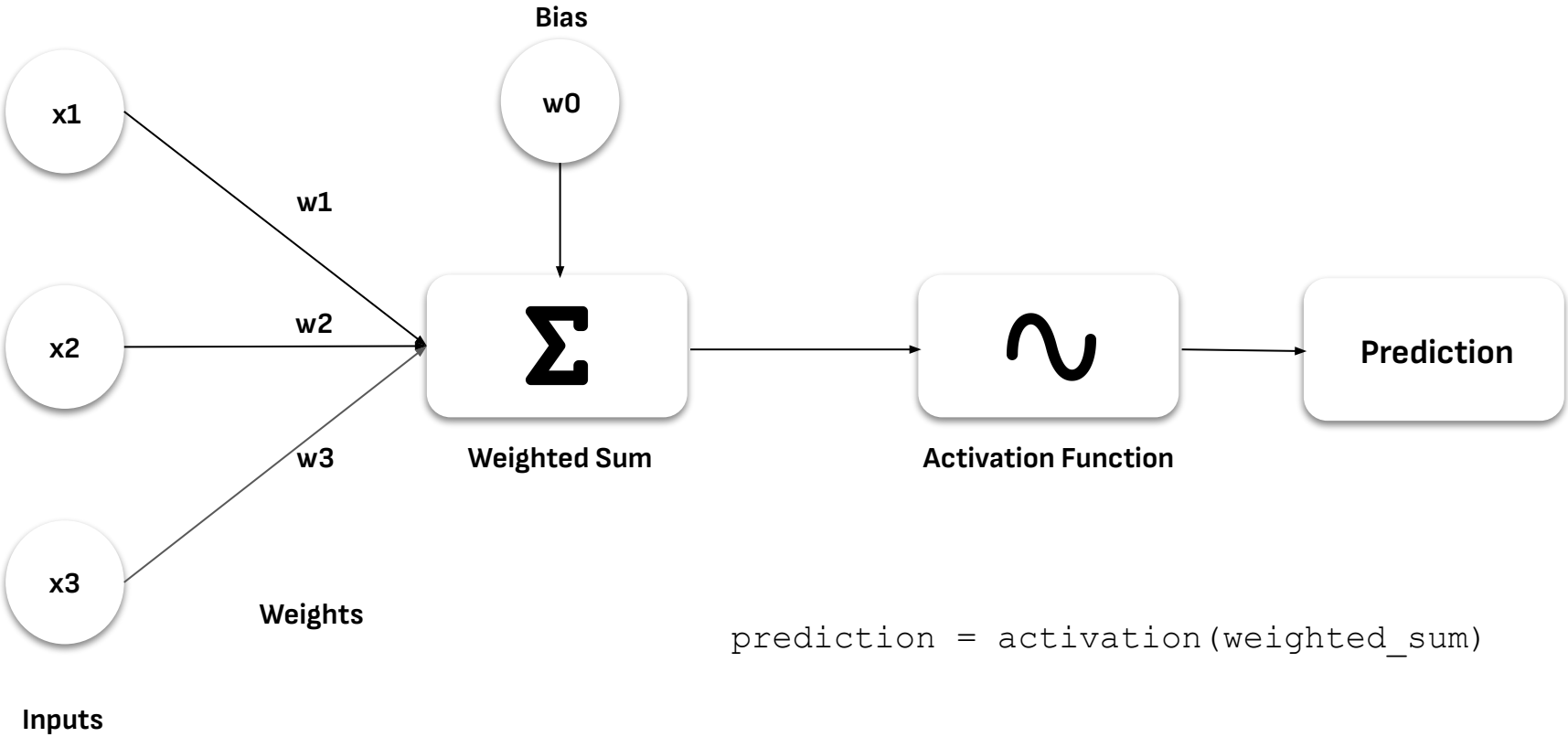

```
weighted_sum = w1x1 + w2x2 + w3x3
```

A constant is added to handle the scenario of all inputs being zero




```
weighted_sum = w1x1 + w2x2 + w3x3 + w0
```

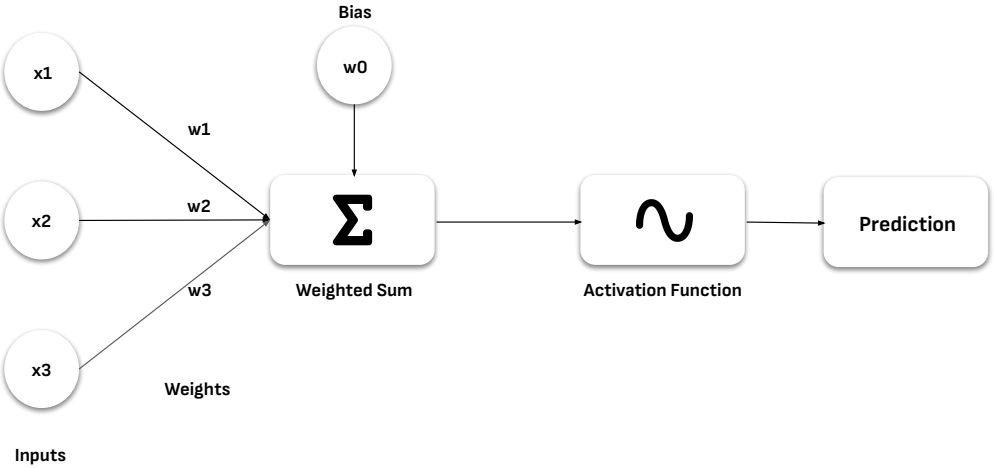
The Weighted Sum is passed through an Activation Function to handle non-linear problems




```
prediction = activation(weighted_sum)
```

Notice that

1. inputs are static
2. weighted_sum and prediction are calculated values.

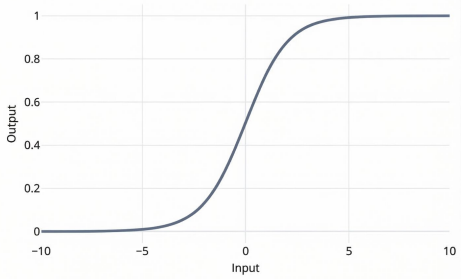


ONLY WEIGHTS AND BIAS ARE UPDATED BY THE MODEL DURING TRAINING.

Most common Activation Functions

Here, x represents the weighted sum

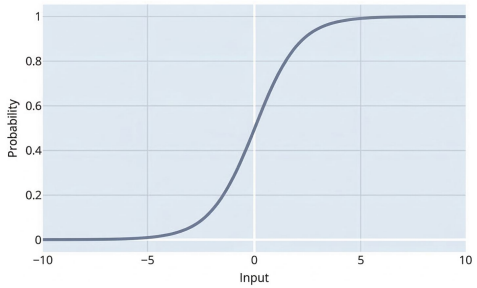
Sigmoid



$$f(x) = \frac{1}{1 + e^{-x}}$$

Squashes input to a value between 0 and 1. Perfect for binary classification probabilities.

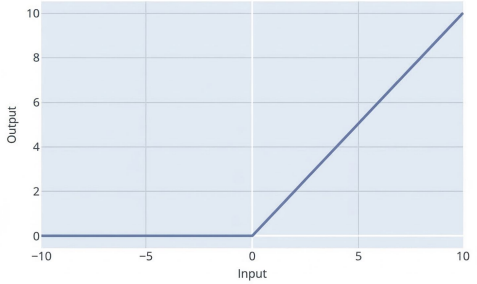
Softmax



$$f(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Outputs a probability distribution that sums to one. Ensures non-negative values for multi-class classification.

ReLU (Rectified Linear Unit)



$$f(x) = \max(0, x)$$

Thresholds negative values to 0. Non-differentiable at x=0, introducing crucial non-linearity.

GeeksforGeeks

The Multi-Layered Perceptron

The Maths of Artificial Neural Network

Perceptrons work best with other perceptrons

The MLP has multiple names

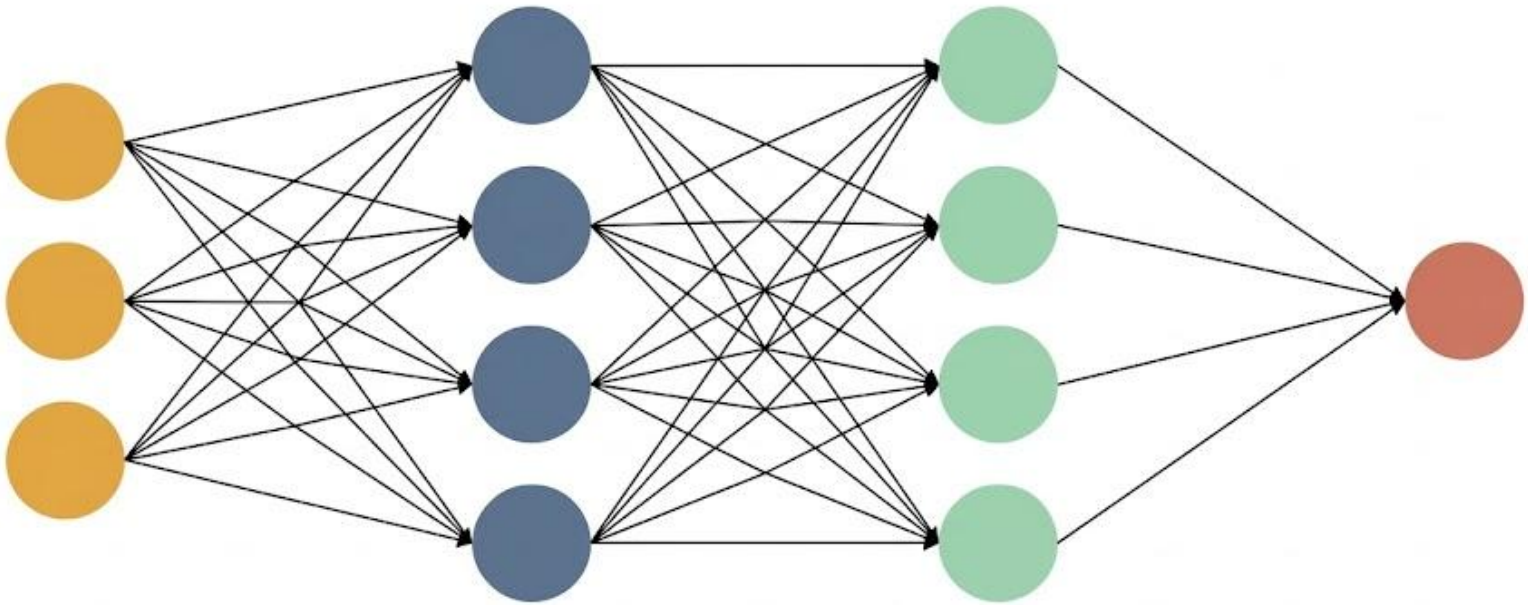
A Layer of Perceptrons

Layer

All perceptrons in a layer collectively perform one specific function or task

This might be receiving input, producing predictions, extracting complex features, or processing hidden patterns

MLP: Multiple Layer of Perceptrons

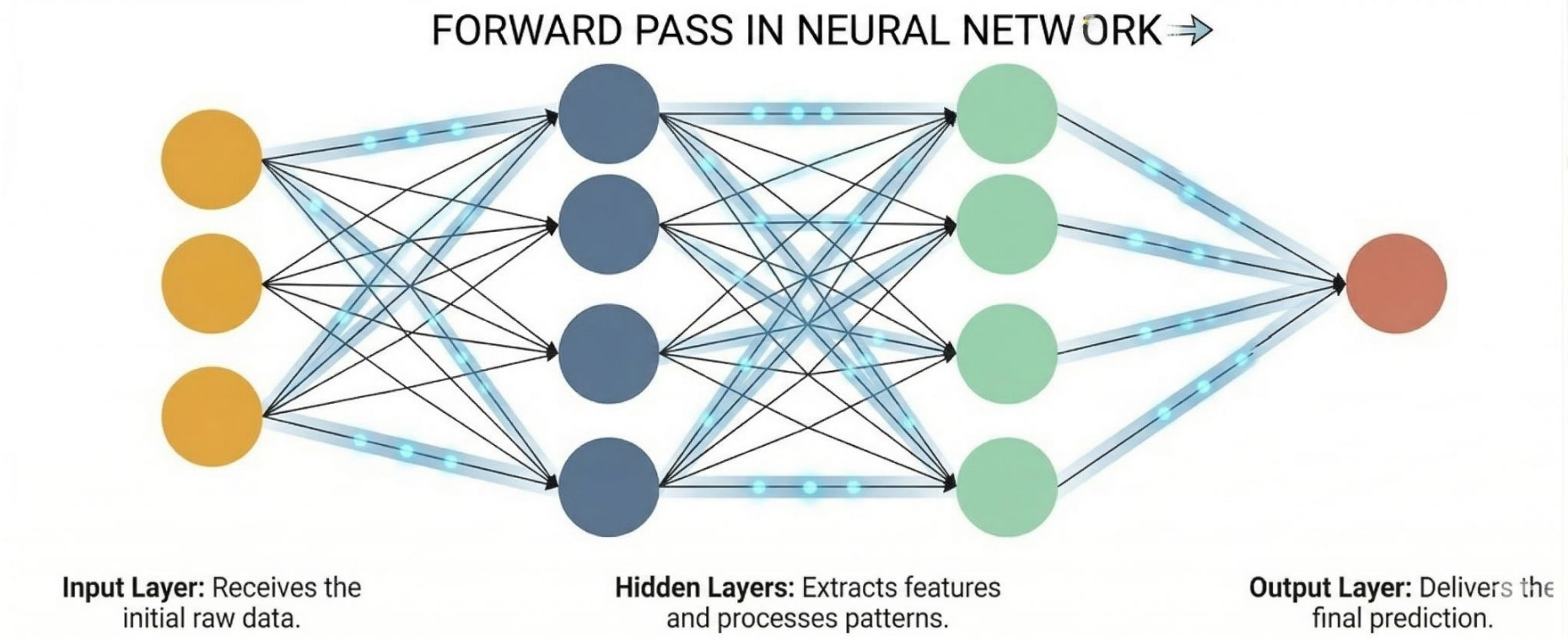


Input Layer: Receives the initial raw data.

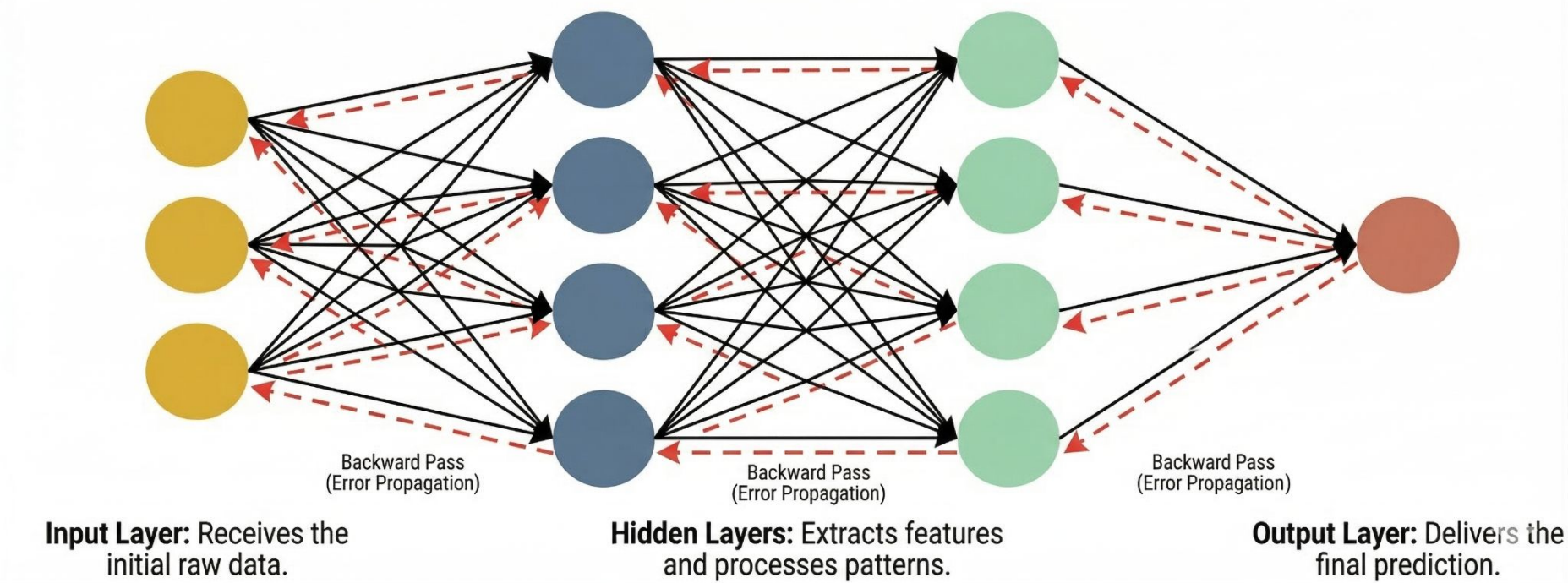
Hidden Layers: Extracts features and processes patterns.

Output Layer: Delivers the final prediction.

The Forward Pass



Backpropagation: The Backward Propagation of Errors

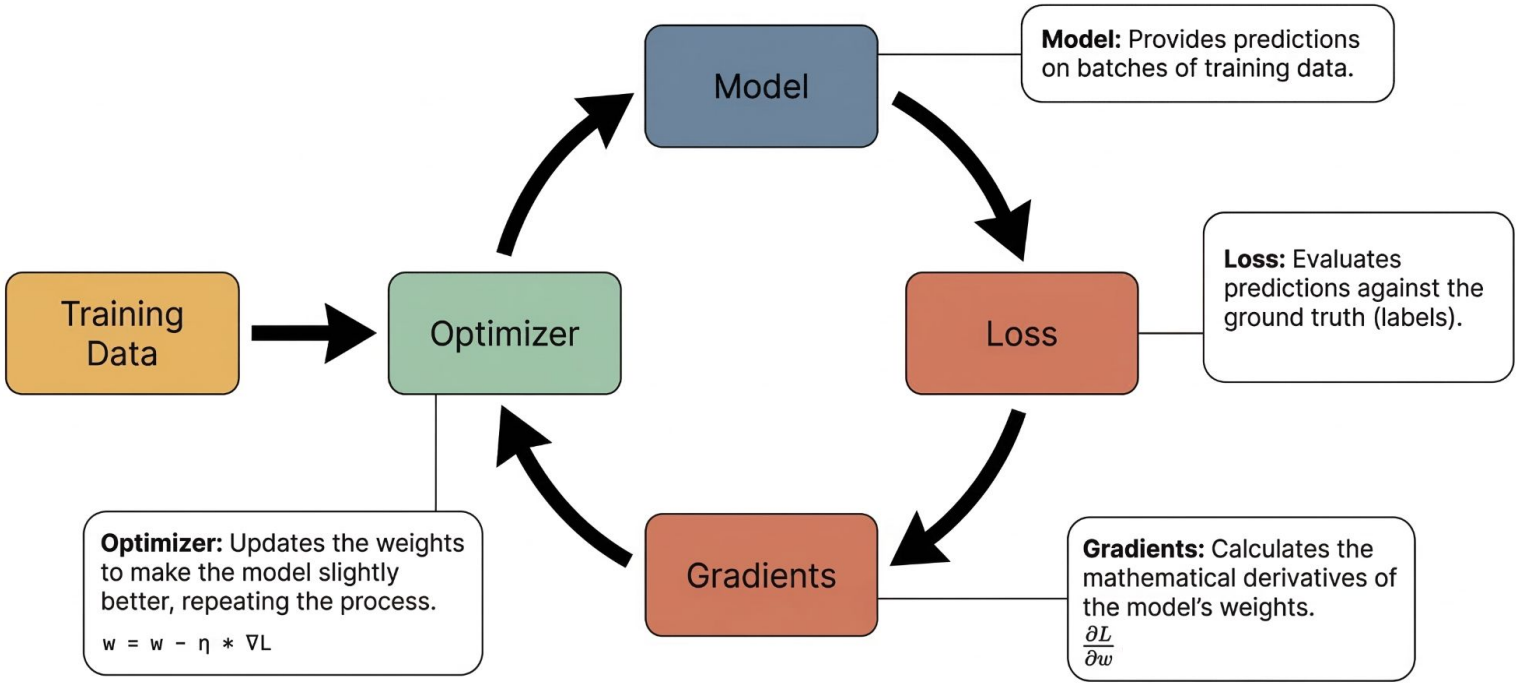


Input Layer: Receives the initial raw data.

Hidden Layers: Extracts features and processes patterns.

Output Layer: Delivers the final prediction.

The For Loop of Training a Neural Network

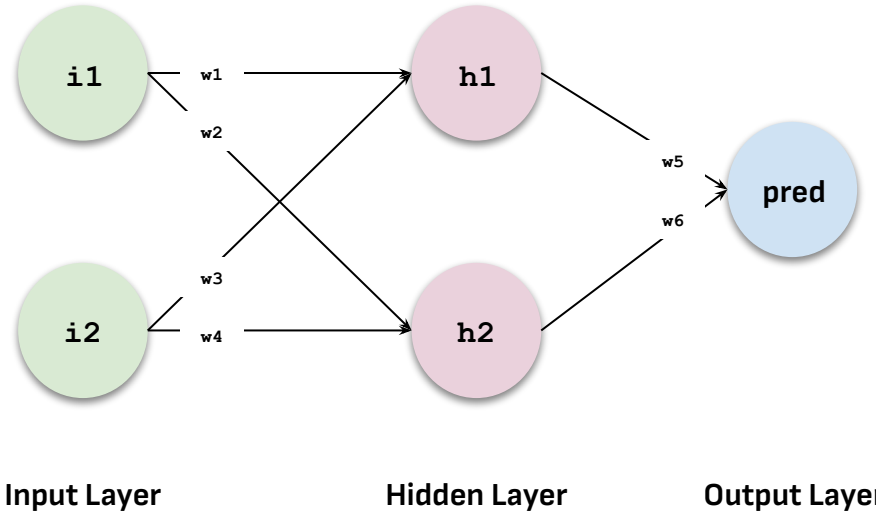


GeeksforGeeks

Training a MLP

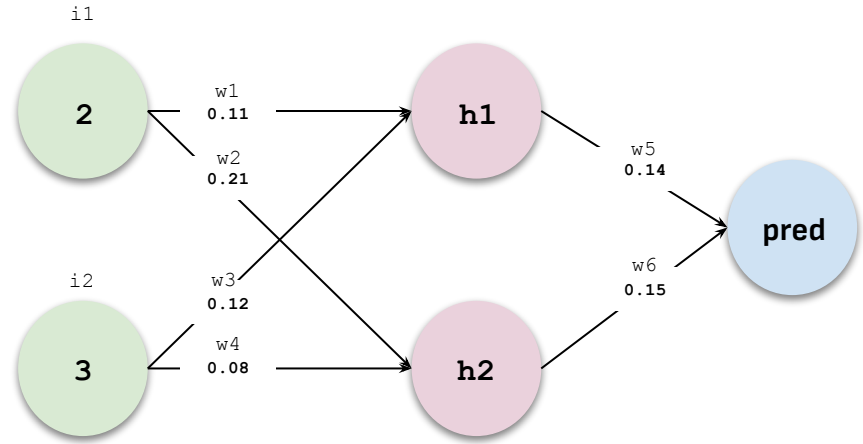
The Maths of Artificial Neural Network

Step 0: Setup



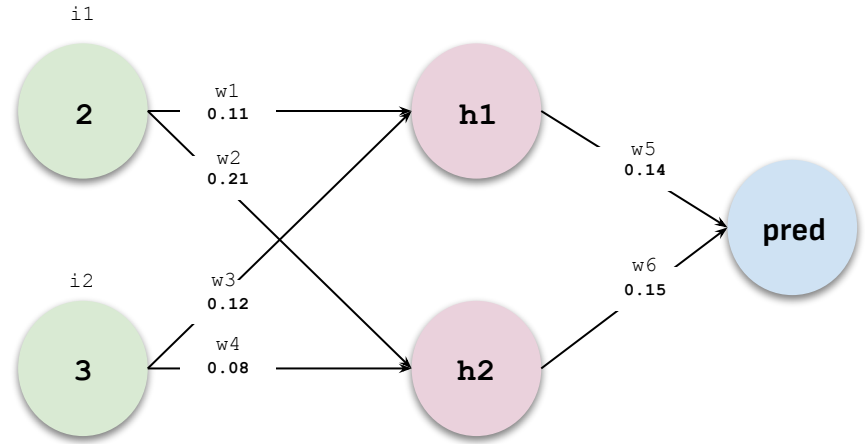
- A Artificial Neural Network with 3 layers:
  - 1 Input Layer with 2 neurons (`i1, i2`)
  - 1 Hidden Layer with 2 neurons (`h1, h2`)
  - 1 Output Layer with 1 neuron (`pred`)
- For the sake of simplicity
  - Each perceptron will not have a bias
  - Each perceptron will not have an activation function

Step 0: Setup



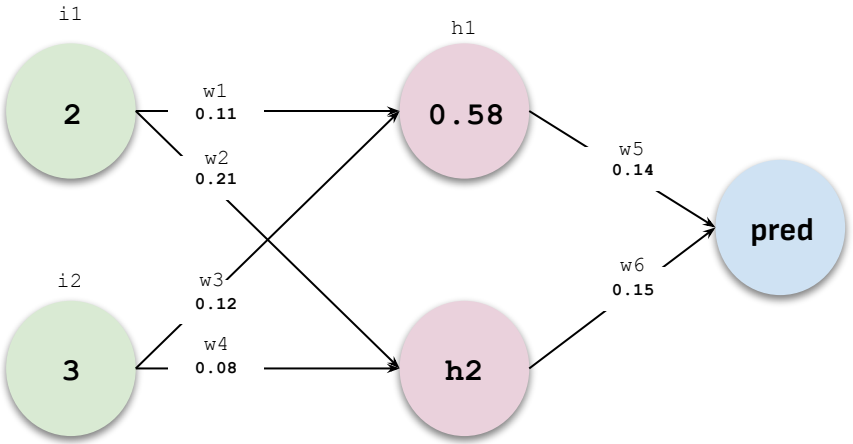

```
i1 = 2
i2 = 3
ground truth (actual) = 1


w1 = 0.11
w2 = 0.21
w3 = 0.12
w4 = 0.08
w5 = 0.14
w6 = 0.15
```

Step 1: Forward Pass



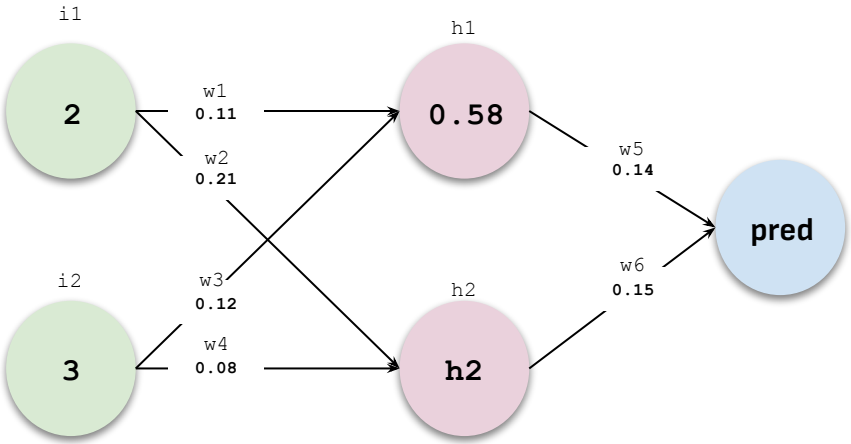

```
h1 = i1 * w1 + i2 * w3
h1 = 2 * 0.11 + 3 * 0.12
h1 = 0.58
```

Step 1: Forward Pass




```
h1 = i1 * w1 + i2 * w3
h1 = 2 * 0.11 + 3 * 0.12
h1 = 0.58
```

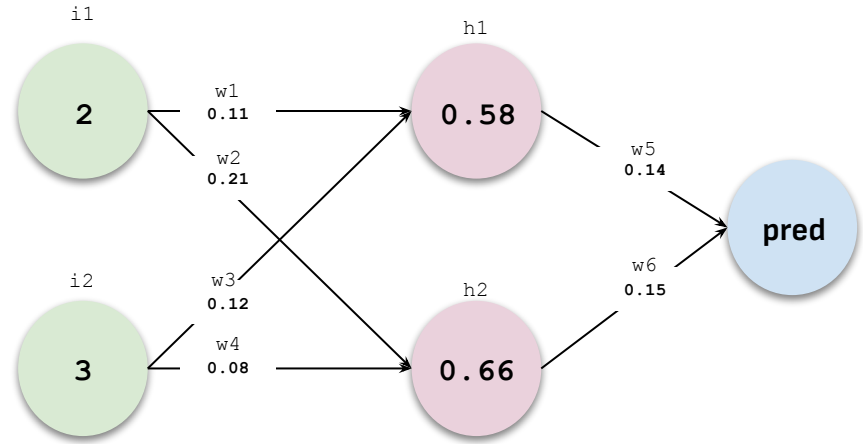
Step 1: Forward Pass




```
h1 = i1 * w1 + i2 * w3
h1 = 2 * 0.11 + 3 * 0.12
h1 = 0.58
```

```
h2 = i1 * w2 + i2 * w4
h2 = 2 * 0.21 + 3 * 0.08
h2 = 0.66
```

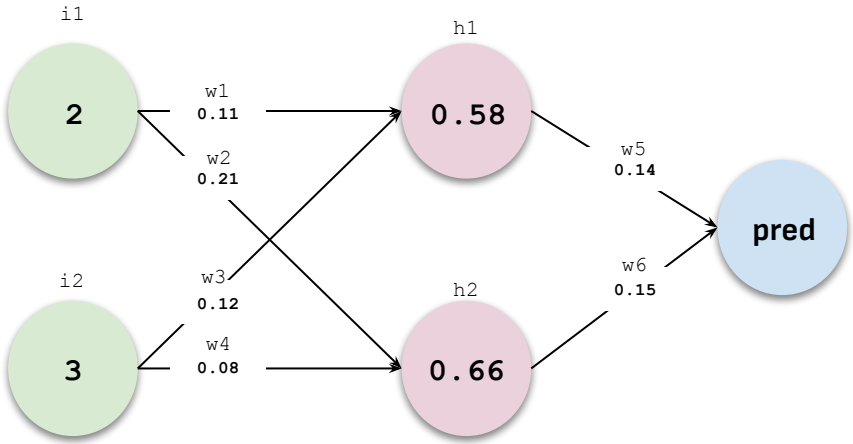
Step 1: Forward Pass




```
h1 = i1 * w1 + i2 * w3
h1 = 2 * 0.11 + 3 * 0.12
h1 = 0.58
```

```
h2 = i1 * w2 + i2 * w4
h2 = 2 * 0.21 + 3 * 0.08
h2 = 0.66
```

Step 1: Forward Pass

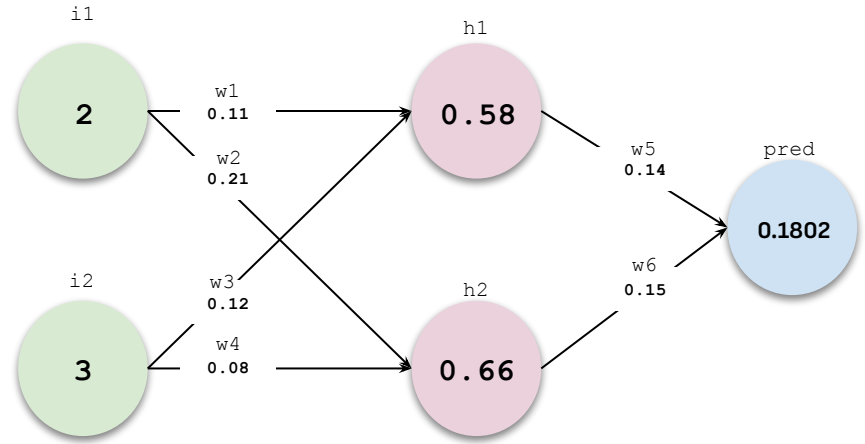



```
h1 = i1 * w1 + i2 * w3
h1 = 2 * 0.11 + 3 * 0.12
h1 = 0.58
```

```
h2 = i1 * w2 + i2 * w4
h2 = 2 * 0.21 + 3 * 0.08
h2 = 0.66
```

```
pred = h1 * w5 + h2 * w6
pred = 0.58 * 0.14 + 0.66 * 0.15
pred = 0.1802
```

Step 1: Forward Pass




```
h1 = i1 * w1 + i2 * w3
h1 = 2 * 0.11 + 3 * 0.12
h1 = 0.58
```

```
h2 = i1 * w2 + i2 * w4
h2 = 2 * 0.21 + 3 * 0.08
h2 = 0.66
```

```
pred = h1 * w5 + h2 * w6
pred = 0.58 * 0.14 + 0.66 * 0.15
pred = 0.1802
```

```
pred = (i1 * w1 + i2 * w3) * w5 + (i1 * w2 +
i2 * w4) * w6
```

Step 2: Error Calculation

$$\text{Error} = \frac{1}{2}(\text{prediction} - \text{actual})^2$$

Notice the

1. square – handles negative errors and penalizes large errors.
2. ½ – just for ease of calculation later.

$$\text{Error} = \frac{1}{2} * (\text{prediction} - \text{actual})^2$$

$$\text{Error} = \frac{1}{2} * (0.1802 - 1)^2$$

$$\text{Error} = \frac{1}{2} * 0.6720$$

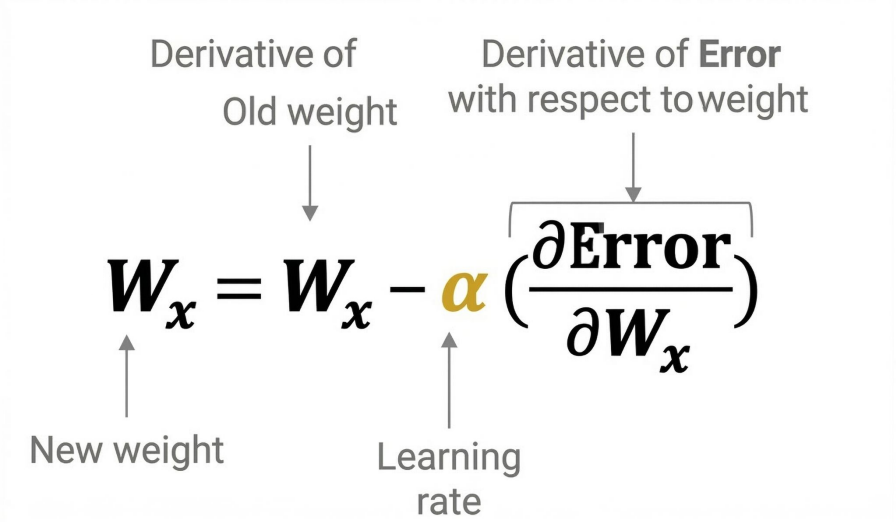
$$\text{Error} = 0.336$$

Step 3: Update Weights using Backpropagation

```
New Weight = Old Weight - Learning Rate * Gradient of Error
```

Let learning rate be `0.05`.

Step 3: Update Weights using Backpropagation



Let learning rate be, that is, `a = 0.05`.

Step 3: Update Weights using Backpropagation

Let's work on w6

$$W_6 = W_6 - \alpha \left(\frac{\partial \text{Error}}{\partial W_6} \right)$$

$$\frac{\partial Error}{\partial W_6} = \frac{\partial \text{Error}}{\partial \text{prediction}} * \frac{\partial prediction}{\partial W_6}$$

chain rule

Error = ½ * (prediction - actual)^2

prediction = h1 * w5 + h2 * w6 = (i1 * w1 + i2 * w3) * w5 + (i1 * w2 + i2 * w4) * w6

Step 3: Update Weights using Backpropagation

Let's work on w6

```
Error = ½ * (prediction - actual)^2

prediction = h1 * w5 + h2 * w6 = (i1 * w1 + i2 * w3) * w5 + (i1 * w2 + i2 * w4) * w6
```

$$\frac{\partial Error}{\partial W_6} = \frac{\partial \frac{1}{2}(\text{prediction} - \text{actual})^2}{\partial prediction} * \frac{[(i_1W_1 + i_2W_3)W_5 + (i_1W_2 + i_2W_4)W_6]}{\partial W_6}$$

$$\frac{\partial Error}{\partial W_6} = 2 * \frac{1}{2}(\text{prediction} - \text{actual}) \frac{\partial(\text{prediction-actual})}{\partial \text{prediciton}} * (i_1w_2 + i_2w_4)$$

Step 3: Update Weights using Backpropagation

Let's work on w6

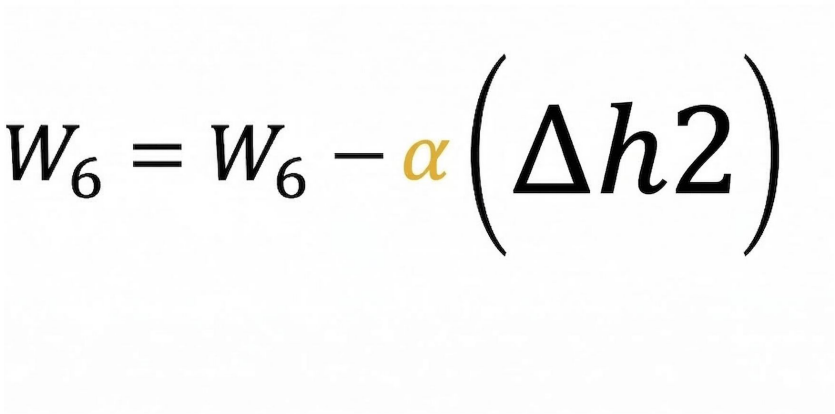
```
h2 = i1 * w2 + i2 * w4

Also, let Δ = prediction - actual
```

$$\frac{\partial Error}{\partial W_6} = (\text{prediction - actual})^* (h_2)$$

$$\frac{\partial Error}{\partial\, w_6} = \Delta h_2$$

Step 3: Update Weights using Backpropagation

Let's work on w6

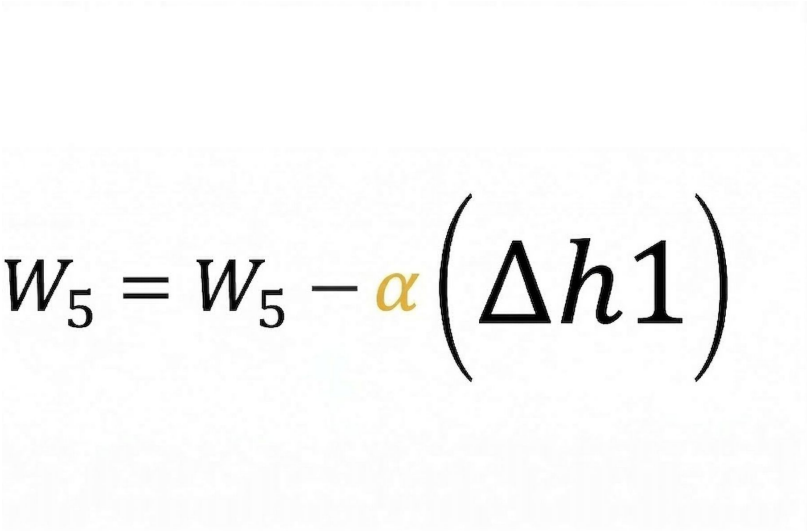


$$W_6 = W_6 - \alpha\left(\Delta h2\right)$$

```
New w6 = Old w6 - α * Δ * h2

New w6 = 0.15 - 0.05 * -0.8198 *
0.66

New w6 = 0.177
```

Step 3: Update Weights using Backpropagation

Let's work on w5



$$W_5 = W_5 - \alpha\left(\Delta h1\right)$$

```
New w5 = Old w5 - α * Δ * h1

New w5 = 0.14 - 0.05 * -0.8198 *
0.58

New w5 = 0.164
```

Step 3: Update Weights using Backpropagation

Let's work on w1

$$\frac{\partial Error}{\partial w_1} = \frac{\partial Error}{\partial \text{prediction}} \cdot \frac{\partial \text{prediction}}{\partial (h_1)} \cdot \frac{\partial (h_1)}{\partial w_1}$$

chain rule

$$\frac{\partial Error}{\partial w_1} = \frac{\partial \left(\frac{1}{2}(\text{prediction} - \text{actual})^2\right)}{\partial \text{predicton}} \cdot \frac{\partial ((h_1)w_5 + (h_2)v_6)}{\partial h_1} \cdot \frac{\partial (i_1 w_1 + i_2 w_3)}{\partial w_1}$$

$$\frac{\partial Error}{\partial w_1} = 2 \cdot \frac{1}{2}(\text{prediction} - \text{actual}) \cdot \frac{\partial (\text{prediction} - \text{actual})}{\partial \text{prediction}} \cdot (w_5) \cdot (i_1)$$

$$\frac{\partial Error}{\partial w_1} = (\text{prediction} - \text{actual}) \cdot (w_5 i_1)$$

$$\frac{\partial Error}{\partial w_1} = \Delta w_5 i_1$$

Step 3: Update Weights using Backpropagation

Let's work on w1

```
New w1 = Old w1 - α * Δ * Old w5 * i1

New w1 = 0.11 - 0.05 * -0.8198 * 0.14 * 2

New w1 = 0.121
```

Step 3: Update Weights using Backpropagation

All weights

$$*w_6 = w_6 - a\,(h_2 \cdot \Delta)$$

$$*w_5 = w_5 - a\,(h_1 \cdot \Delta)$$

$$*w_4 = w_4 - a\,(i_2 \cdot \Delta w_6)$$

$$*w_3 = w_3 - a\,(i_2 \cdot \Delta \cdot w_5)$$

$$*w_2 = w_2 - a\,(i_1 \cdot \Delta w_6)$$

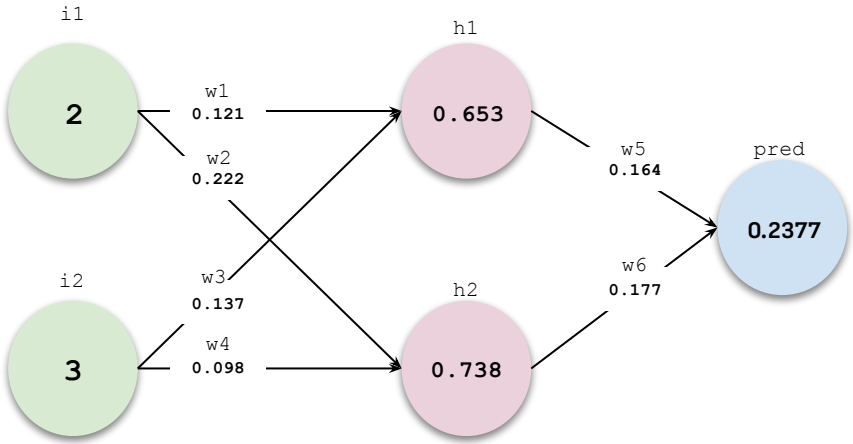
$$*w_1 = w_1 - a\,(i_1 \cdot \Delta w_5)$$

Step 3: Update Weights using Backpropagation

All weights

```
New w6 = 0.177
New w5 = 0.164
New w4 = 0.098
New w3 = 0.137
New w2 = 0.222
New w1 = 0.121
```

Iteration 2: Step 1 Again: Forward Pass




```
New h1 = i1 * New w1 + i2 * New w3
New h1 = 2 * 0.121 + 3 * 0.137
New h1 = 0.653
```

```
New h2 = i1 * New w2 + i2 * New w4
New h2 = 2 * 0.222 + 3 * 0.098
New h2 = 0.738
```

```
New pred = New h1 * New w5 + New h2 * New w6
New pred = 0.653 * 0.164 + 0.738 * 0.177
New pred = 0.2377
```

Step 2 Again: Error Calculation

$$\text{Error} = \frac{1}{2}(\text{prediction} - \text{actual})^2$$

$$\text{New Error} = \frac{1}{2} * (\text{New prediction} - \text{actual})^2$$

$$\text{New Error} = \frac{1}{2} * (0.2377 - 1)^2$$

$$\text{New Error} = \frac{1}{2} * 0.5811$$

$$\text{New Error} = 0.290$$

The error reduces from 0.336 to 0.290!!

This continues until error becomes zero

or user-defined iteration / time limit is reached.

CONGRATULATIONS!! You just learned Backpropagation via

GRADIENT DESCENT

1. Learning Rate
2. Epochs
3. Batch Size